



IBM Developer  
SKILLS NETWORK

# Winning Space Race with Data Science

Roshini S

04-Jan-2026



# Outline

---

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

# Executive Summary

---

- Summary of methodologies
  - Data Collection through API
  - Data Collection with Web Scraping
  - Data Wrangling
  - Exploratory Data Analysis with SQL
  - Exploratory Data Analysis with Data Visualization
  - Interactive Visual Analytics with Folium
  - Machine Learning Prediction
- Summary of all results
  - Exploratory Data Analysis result
  - Interactive analytics in screenshots
  - Predictive Analytics result from Machine Learning Lab

# Introduction

---

SpaceX is a revolutionary company who has disrupted the space industry by offering a rocket launch specifically Falcon 9 as low as 62 million dollars; while other providers cost upward of 165 million dollars each. Most of this saving thanks to SpaceX's astounding idea to reuse the first stage of the launch by re-land the rocket to be used on the next mission. Repeating this process will make the price even further down. As a data scientist of a startup rivaling SpaceX, the goal of this project is to create the machine learning pipeline to predict the landing outcome of the first stage in the future. This project is crucial in identifying the right price to bid against SpaceX for a rocket launch.

The problems included:

- Identifying all factors that influence the landing outcome.
- The relationship between each variable and how it is affecting the outcome.
- The best condition needed to increase the probability of successful landing.





Section 1

# Methodology

# Methodology

---

## Executive Summary

- Data collection methodology:
  - Data was collected using SpaceX REST API and web scrapping from Wikipedia
- Perform data wrangling
  - Data was processed using one-hot encoding for categorical features
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
  - How to build, tune, evaluate classification models

# Data Collection

---

Data collection is the process of gathering and measuring information on targeted variables in an established system, which then enables one to answer relevant questions and evaluate outcomes. As mentioned, the dataset was collected by REST API and Web Scrapping from Wikipedia

For REST API, its started by using the get request. Then, we decoded the response content as Json and turn it into a pandas dataframe using `json_normalize()`. We then cleaned the data, checked for missing values and fill with whatever needed.

For web scrapping, we will use the BeautifulSoup to extract the launch records as HTML table, parse the table and convert it to a pandas dataframe for further analysis

# Data Collection - SpaceX API

Get request for rocket launch data using API

Use json\_normalize method to convert json result to dataframe

Performed data cleaning and filling the missing value

From:

[https://github.com/Roshz28/IBM-capstone/blob/main/01\\_SpaceX\\_Data\\_Collection\\_API.ipynb](https://github.com/Roshz28/IBM-capstone/blob/main/01_SpaceX_Data_Collection_API.ipynb)

```
spacex_url="https://api.spacexdata.com/v4/launches/past"
```

```
response = requests.get(spacex_url)
```

```
# Use json_normalize method to convert the json result into a dataframe  
data = pd.json_normalize(response.json())
```

```
# Lets take a subset of our dataframe keeping only the features we want and the flight number, and date_utc.  
data = data[['rocket', 'payloads', 'launchpad', 'cores', 'flight_number', 'date_utc']]
```

```
# We will remove rows with multiple cores because those are falcon rockets with 2 extra rocket boosters and rows that have  
data = data[data['cores'].map(len)==1]  
data = data[data['payloads'].map(len)==1]
```

```
# Since payloads and cores are lists of size 1 we will also extract the single value in the list and replace the feature.  
data['cores'] = data['cores'].map(lambda x : x[0])  
data['payloads'] = data['payloads'].map(lambda x : x[0])
```

```
# We also want to convert the date_utc to a datetime datatype and then extracting the date leaving the time  
data['date'] = pd.to_datetime(data['date_utc']).dt.date
```

```
# Using the date we will restrict the dates of the launches  
data = data[data['date'] <= datetime.date(2020, 11, 13)]
```



# Data Collection - Scraping

---

Request the Falcon9  
Launch Wiki page from url

Create a BeautifulSoup  
from the HTML response

Extract all column/variable  
names from the HTML  
header

From:

[https://github.com/Roshz28/IBM-capstone/blob/main/02\\_SpaceX\\_Web\\_Scraping.ipynb](https://github.com/Roshz28/IBM-capstone/blob/main/02_SpaceX_Web_Scraping.ipynb)

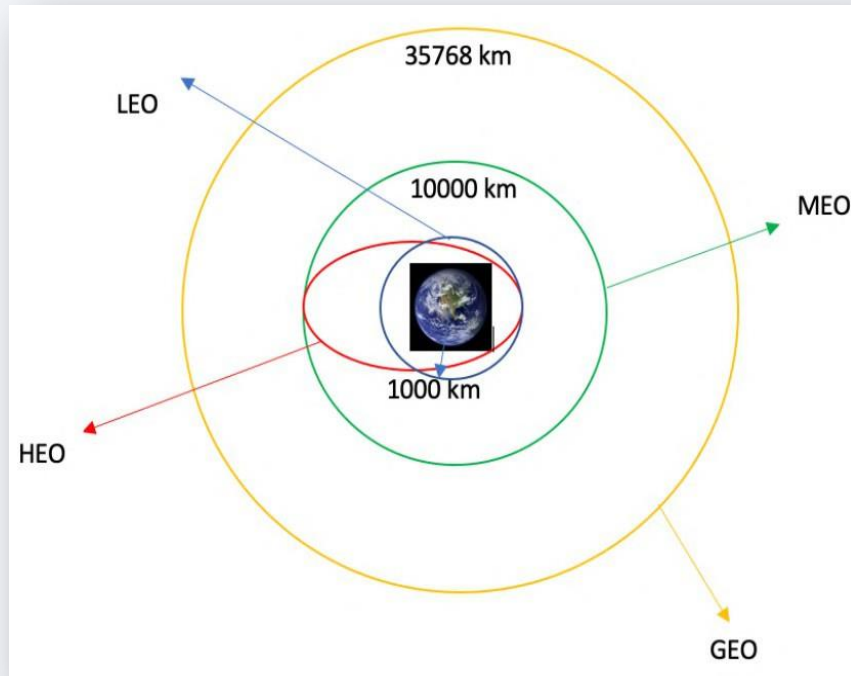
```
# use requests.get() method with the provided static_url
# assign the response to a object
data = requests.get(static_url).text
```

```
# Use BeautifulSoup() to create a BeautifulSoup object from a response text content
soup = BeautifulSoup(data, 'html.parser')
```

```
extracted_row = 0
#Extract each table
for table_number, table in enumerate(soup.find_all('table', "wikitable plainrowheaders collapsible")):
    # get table row
    for rows in table.find_all("tr"):
        #check to see if first table heading is as number corresponding to launch a number
        if rows.th:
            if rows.th.string:
                flight_number=rows.th.string.strip()
                flag=flight_number.isdigit()
            else:
                flag=False
```

# Data Wrangling

---



From:

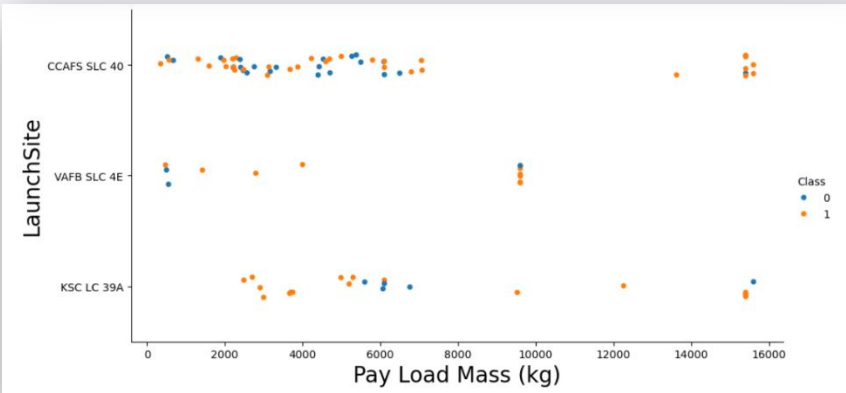
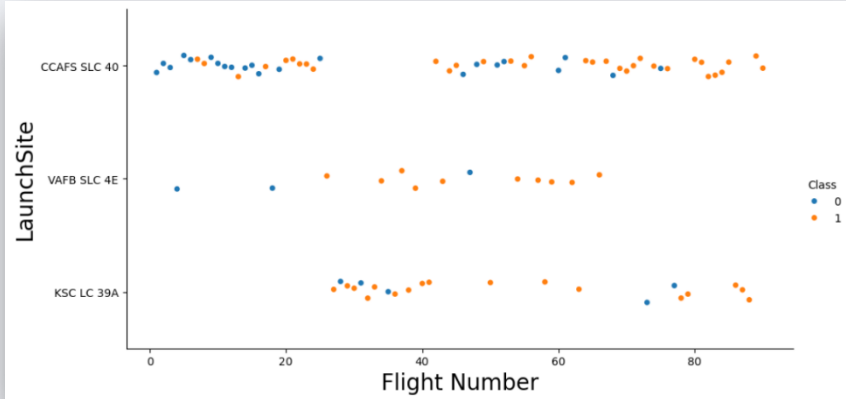
[https://github.com/Roshz28/IBM-capstone/blob/main/03\\_SpaceX\\_Data\\_Wrangling.ipynb](https://github.com/Roshz28/IBM-capstone/blob/main/03_SpaceX_Data_Wrangling.ipynb)

Data Wrangling is the process of cleaning and unifying messy and complex data sets for easy access and Exploratory Data Analysis (EDA).

We will first calculate the number of launches on each site, then calculate the number and occurrence of mission outcome per orbit type.

We then create a landing outcome label from the outcome column. This will make it easier for further analysis, visualization, and ML. Lastly, we will export the result to a CSV.

# EDA with Data Visualization



We first started by using scatter graph to find the relationship between the attributes such as between:

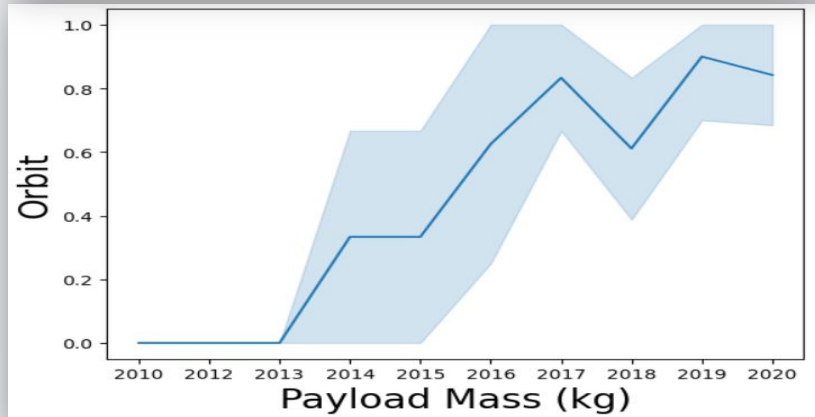
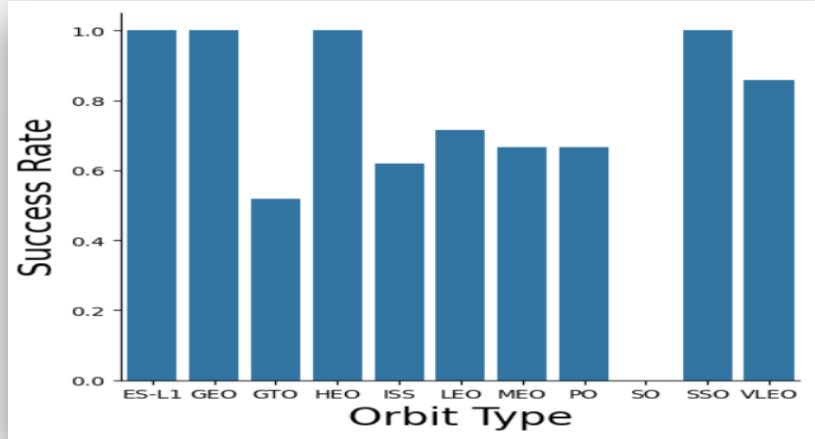
- Flight Number and Launch Site.
- Payload and Launch Site.
- Success Rate of Each Orbit.
- Flight Number and Orbit Type.
- Payload and Orbit Type.
- Launch Success Yearly Trend

Scatter plots show dependency of attributes on each other. Once a pattern is determined from the graphs. It's very easy to see which factors affecting the most to the success of the landing outcomes.

From:

[https://github.com/Roshz28/IBM-capstone/blob/main/05\\_SpaceX\\_EDA\\_Data\\_Visualization.ipynb](https://github.com/Roshz28/IBM-capstone/blob/main/05_SpaceX_EDA_Data_Visualization.ipynb)

# EDA with Data Visualization



Once we get a hint of the relationships using scatter plot. We will then use further visualization tools such as bar graph and line plots graph for further analysis.

Bar graphs is one of the easiest way to interpret the relationship between the attributes. In this case, we will use the bar graph to determine which orbits have the highest probability of success.

We then use the line graph to show a trends or pattern of the attribute over time which in this case, is used for see the launch success yearly trend.

We then use Feature Engineering to be used in success prediction in the future module by created the dummy variables to categorical columns.

From:

[https://github.com/Roshz28/IBM-capstone/blob/main/05\\_SpaceX\\_EDA\\_Data\\_Visualization.ipynb](https://github.com/Roshz28/IBM-capstone/blob/main/05_SpaceX_EDA_Data_Visualization.ipynb)

# EDA with SQL

---

Using SQL, we had performed many queries to get better understanding of the dataset, Ex:

- Displaying the names of the launch sites.
- Displaying 5 records where launch sites begin with the string 'CCA'.
- Displaying the total payload mass carried by booster launched by NASA (CRS).
- Displaying the average payload mass carried by booster version F9 v1.1.
- Listing the date when the first successful landing outcome in ground pad was achieved.
- Listing the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000.
- Listing the total number of successful and failure mission outcomes.
- Listing the names of the booster\_versions which have carried the maximum payload mass.
- Listing the failed landing\_outcomes in drone ship, their booster versions, and launch sites names for in year 2015.
- Rank the count of landing outcomes or success between the date 2010-06-04 and 2017-03-20, in descending order.

From:

[https://github.com/Roshz28/IBM-capstone/blob/main/04\\_SpaceX\\_EDA\\_SQL.ipynb](https://github.com/Roshz28/IBM-capstone/blob/main/04_SpaceX_EDA_SQL.ipynb)



# Build an Interactive Map with Folium

---

To visualize the launch data into an interactive map. We took the latitude and longitude coordinates at each launch site and added a circle marker around each launch site with a label of the name of the launch site.

We then assigned the dataframe `launch_outcomes(failure,success)` to classes 0 and 1 with **Red** and **Green** markers on the map in `MarkerCluster()`.

We then used the Haversine's formula to calculate the distance of the launch sites to various landmarks to find answers to the questions of:

- How close the launch sites with railways, highways and coastlines?
- How close the launch sites with nearby cities?

From:

[https://github.com/Roshz28/IBM-capstone/blob/main/06\\_SpaceX\\_Interactive\\_Visual\\_Analytics\\_Folium.ipynb](https://github.com/Roshz28/IBM-capstone/blob/main/06_SpaceX_Interactive_Visual_Analytics_Folium.ipynb)

# Build a Dashboard with Plotly Dash

---

- We built an interactive dashboard with Plotly dash which allowing the user to play around with the data as they need.
- We plotted pie charts showing the total launches by a certain sites.
- We then plotted scatter graph showing the relationship with Outcome and Payload Mass (Kg) for the different booster version.

From:

[https://github.com/Roshz28/IBM-capstone/blob/main/07\\_SpaceX\\_Interactive\\_Visual\\_Analytics\\_Plotly.py](https://github.com/Roshz28/IBM-capstone/blob/main/07_SpaceX_Interactive_Visual_Analytics_Plotly.py)

# Predictive Analysis (Classification)

---

## Building the Model

- Load the dataset into NumPy and Pandas
- Transform the data and then split into training and test datasets
- Decide which type of ML to use
- set the parameters and algorithms to GridSearchCV and fit it to dataset.

## Evaluating the Model

- Check the accuracy for each model
- Get tuned hyperparameters for each type of algorithms.
- plot the confusion matrix.

## Improving the Model

- Use Feature Engineering and Algorithm Tuning

## Find the Best Model

- The model with the best accuracy score will be the best performing model.

From:

[https://github.com/Roshz28/IBM-capstone/blob/main/08\\_SpaceX\\_Predictive\\_Analytics.ipynb](https://github.com/Roshz28/IBM-capstone/blob/main/08_SpaceX_Predictive_Analytics.ipynb)

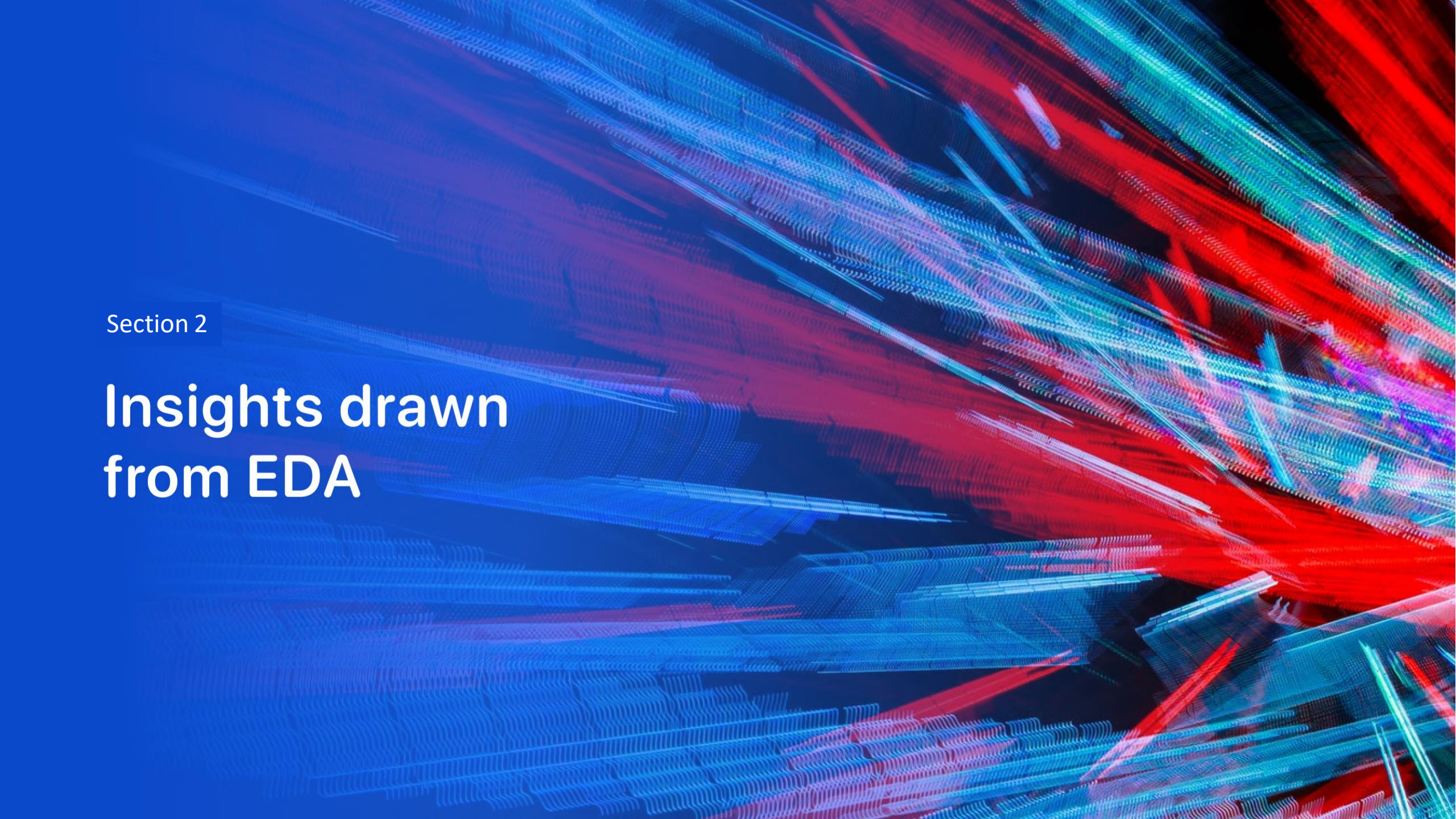
# Results

---

The results will be categorized to 3 main results which is:

- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results



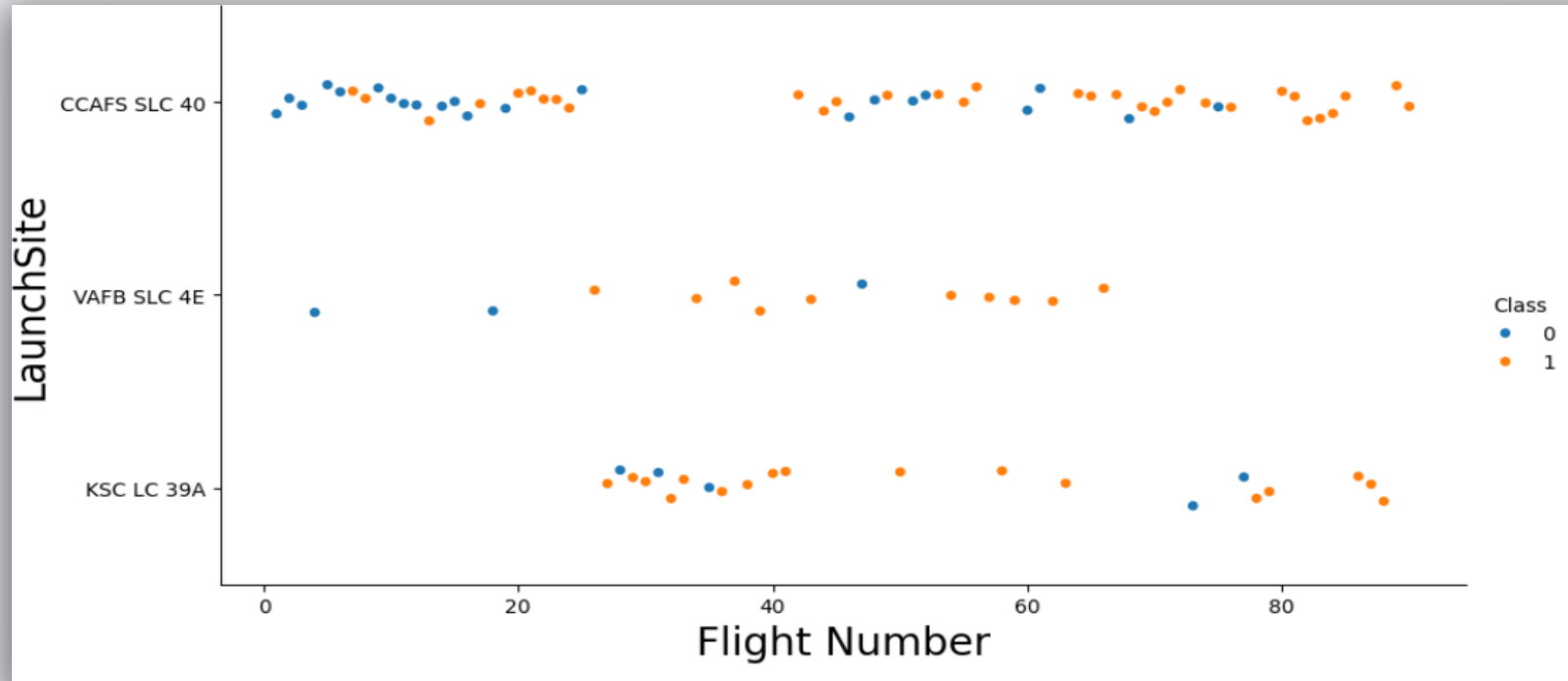
The background of the slide is an abstract composition. It features a dark blue base color. Overlaid on this are numerous diagonal streaks in shades of red and cyan. A faint, light blue grid pattern is also visible, particularly in the lower half of the image. The overall effect is dynamic and technological.

Section 2

# Insights drawn from EDA



# Flight Number vs. Launch Site



This scatter plot shows that the larger the flights amount of the launch site, the greater the success rate will be.

However, site CCAFS SLC40 shows the least pattern of this.

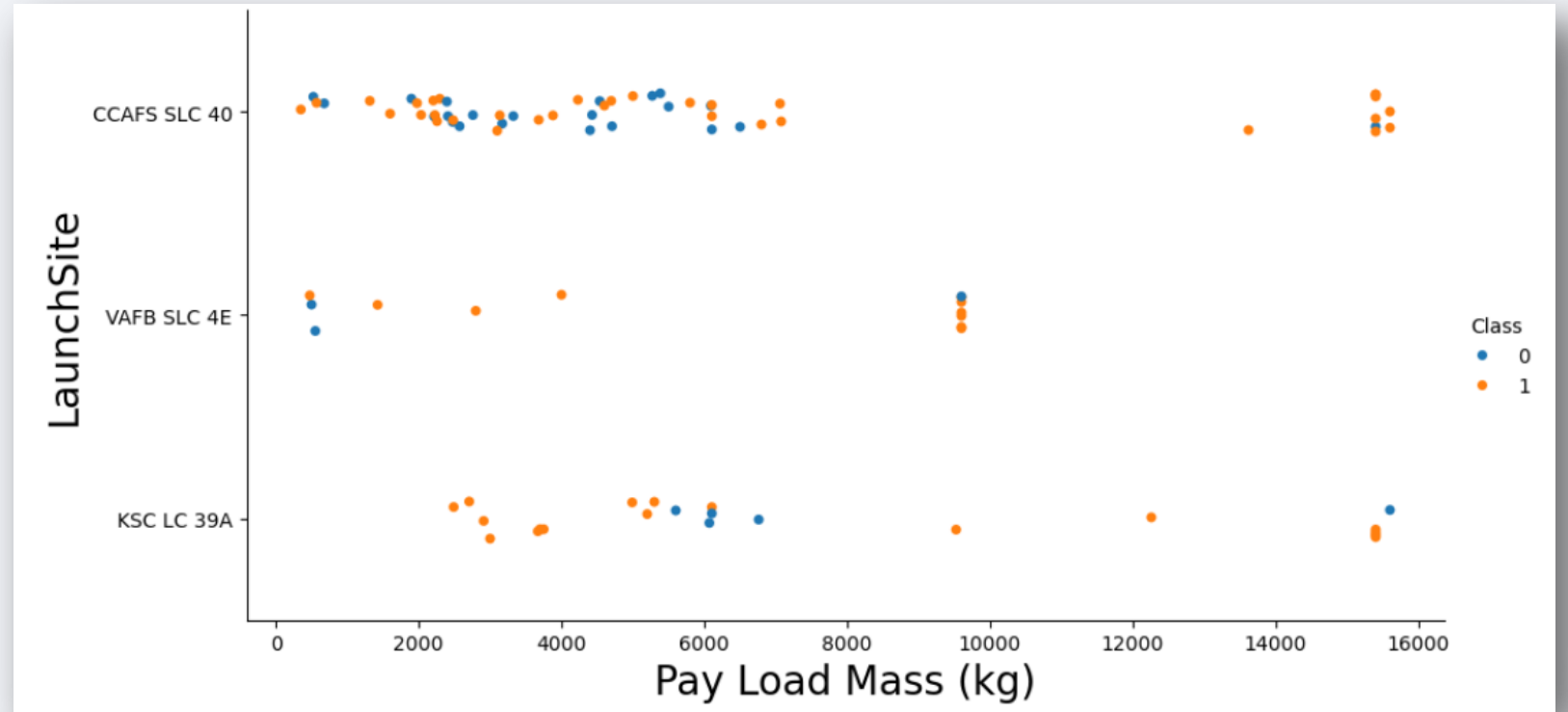


# Payload vs. Launch Site

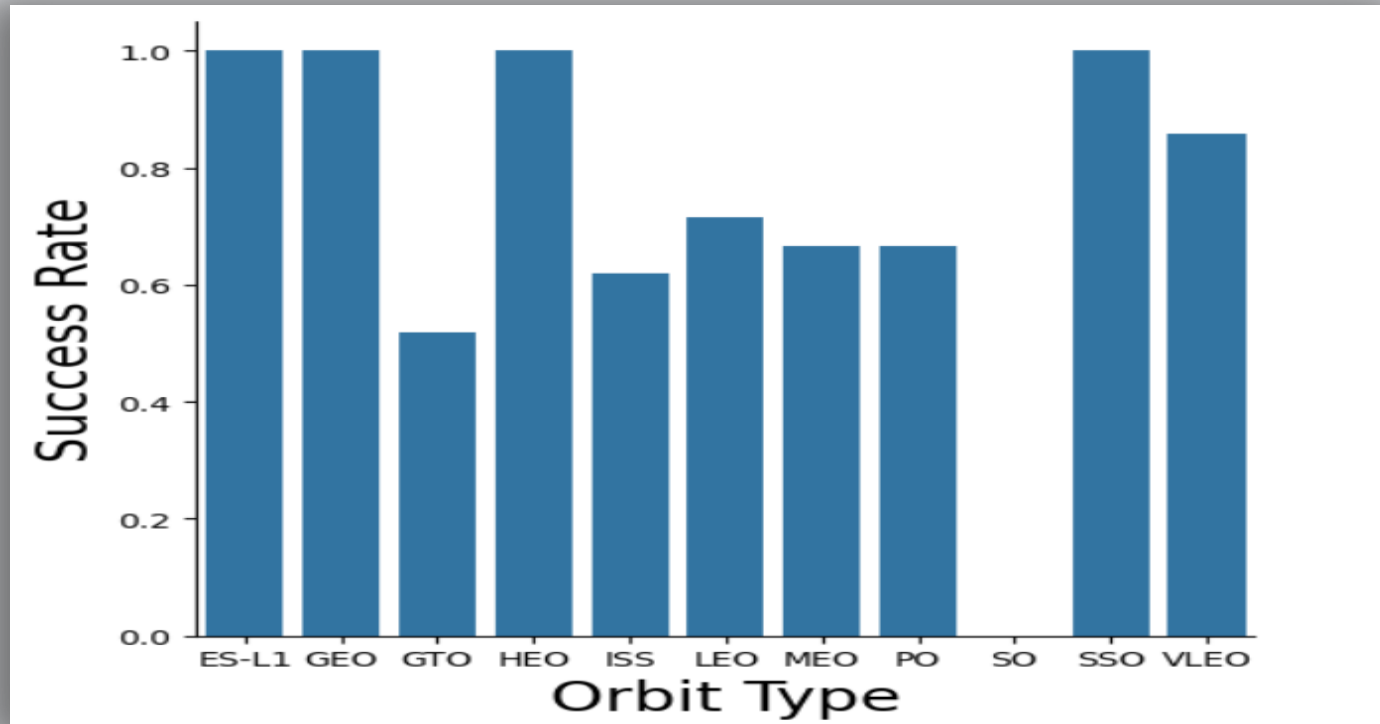
This scatter plot shows once the payload mass is greater than 7000kg, the probability of the success rate will be highly increased.

However, there is no clear pattern to say the launch site is dependent to the payload mass for the success rate

VAFB-SLC launchsite there are no rockets launched for heavy payload mass (greater than 10000).



# Success Rate vs. Orbit Type



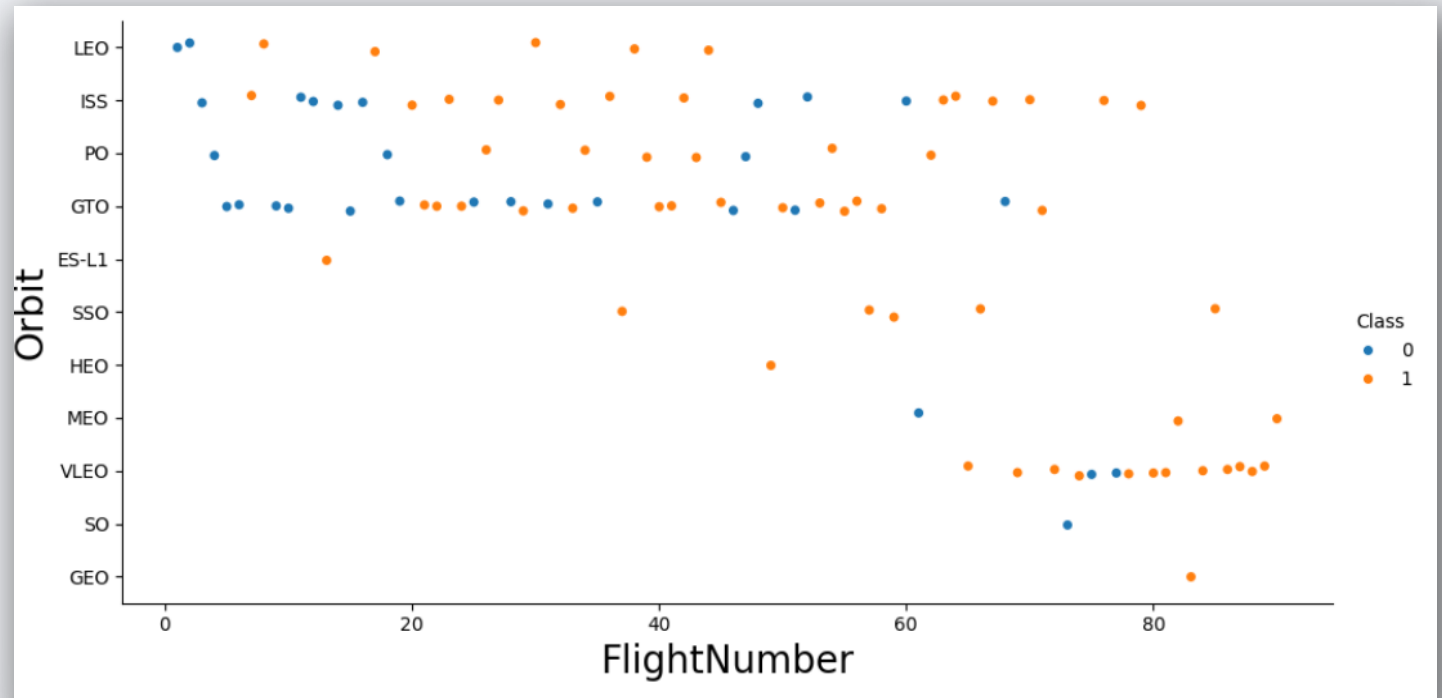
This figure depicted the possibility of the orbits to influences the landing outcomes as some orbits has 100% success rate such as SSO, HEO, GEO AND ES-L1 while SO orbit produced 0% rate of success.

However, deeper analysis show that some of this orbits has only 1 occurrence such as GEO, SO, HEO and ES-L1 which mean this data need more dataset to see pattern or trend before we draw any conclusion.

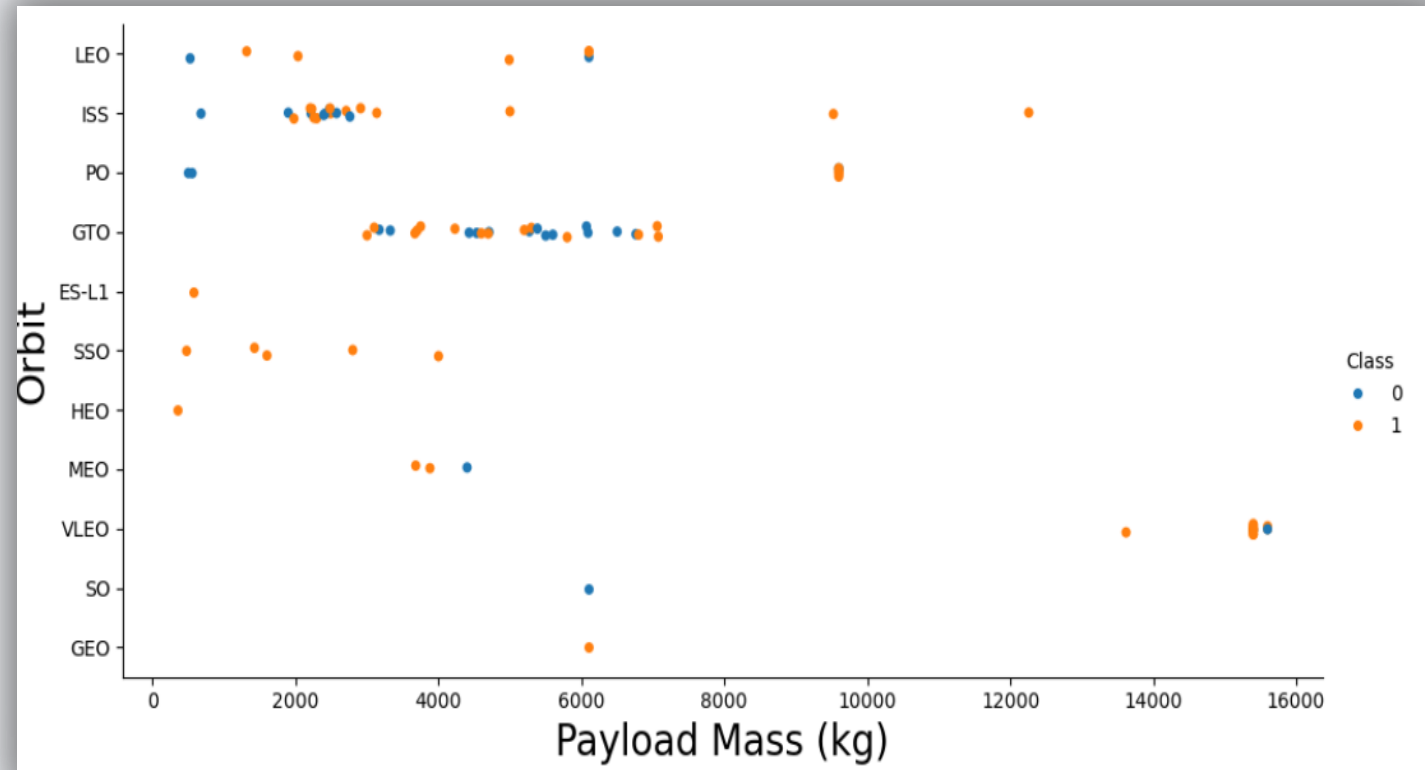
# Flight Number vs. Orbit Type

This scatter plot shows that generally, the larger the flight number on each orbits, the greater the success rate (especially LEO orbit) except for GTO orbit which depicts no relationship between both attributes.

Orbit that only has 1 occurrence should also be excluded from above statement as it's needed more dataset.



# Payload vs. Orbit Type



Heavier payload has positive impact on LEO, ISS and PO orbit. However, it has negative impact on MEO and VLEO orbit.

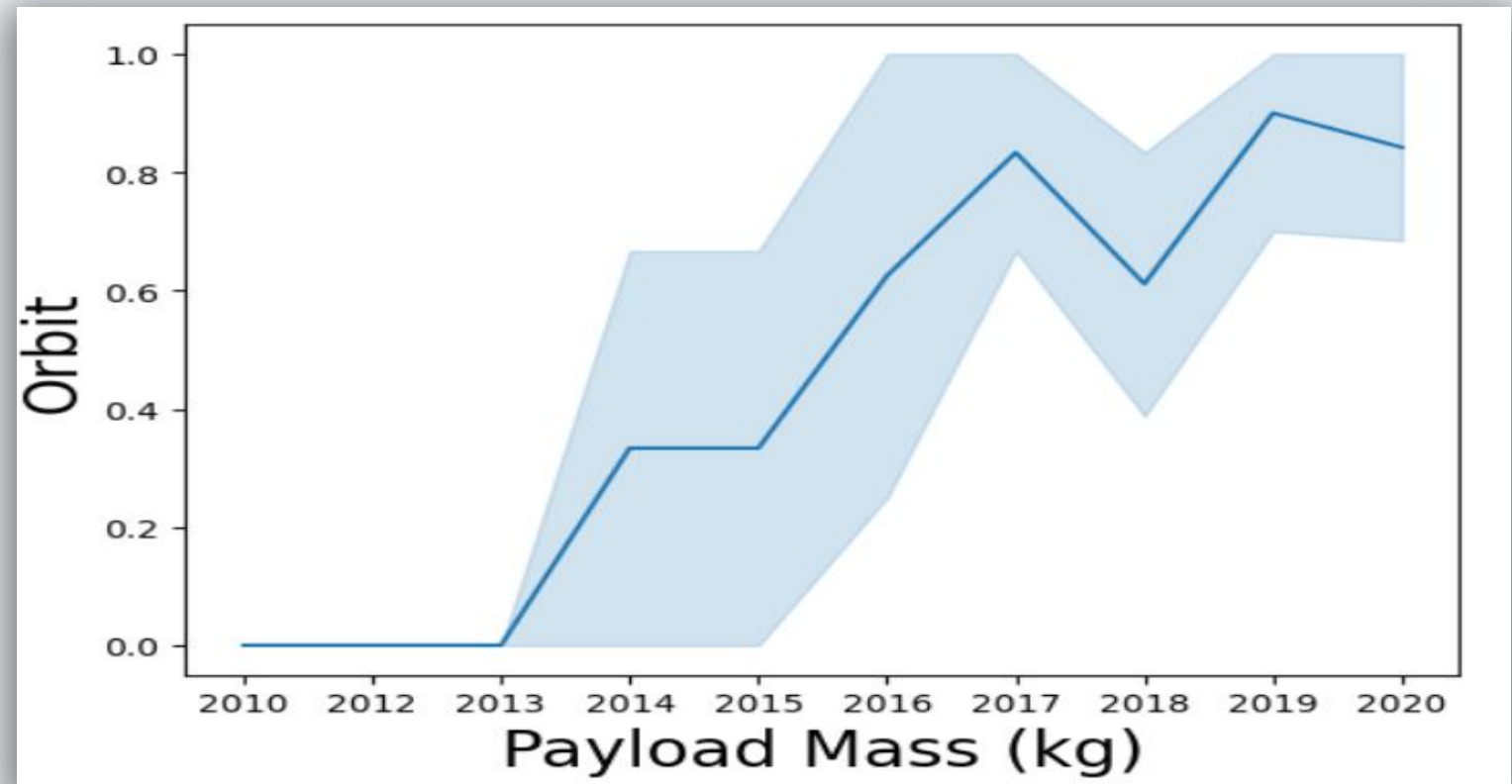
GTO orbit seem to depict no relation between the attributes.

Meanwhile, again, SO, GEO and HEO orbit need more dataset to see any pattern or trend.

# Launch Success Yearly Trend

This figures clearly depicted and increasing trend from the year 2013 until 2020.

If this trend continue for the next year onward. The success rate will steadily increase until reaching 1/100% success rate.





# All Launch Site Names

---

We used the key word **DISTINCT** to show only unique launch sites from the SpaceX data.

```
In [10]: %sql SELECT distinct(Launch_Site) FROM SPACEXTBL;
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
Out[10]: Launch_Site
```

```
CCAFS LC-40
```

```
VAFB SLC-4E
```

```
KSC LC-39A
```

```
CCAFS SLC-40
```

# Launch Site Names Begin with 'CCA'

We used the query above to display 5 records where launch sites begin with `CCA`

```
In [12]: %sql SELECT * FROM SPACEXTBL WHERE Launch_Site LIKE 'CCA%' LIMIT 5;
```

\* sqlite:///my\_data1.db  
Done.

```
Out[12]:
```

| Date       | Time (UTC) | Booster_Version | Launch_Site | Payload                                                       | PAYLOAD_MASS_KG_ | Orbit     | Customer        | Mission_Outcome | Landing_Outc    |
|------------|------------|-----------------|-------------|---------------------------------------------------------------|------------------|-----------|-----------------|-----------------|-----------------|
| 2010-06-04 | 18:45:00   | F9 v1.0 B0003   | CCAFS LC-40 | Dragon Spacecraft Qualification Unit                          | 0                | LEO       | SpaceX          | Success         | Failure (paracl |
| 2010-12-08 | 15:43:00   | F9 v1.0 B0004   | CCAFS LC-40 | Dragon demo flight C1, two CubeSats, barrel of Brouere cheese | 0                | LEO (ISS) | NASA (COTS) NRO | Success         | Failure (paracl |
| 2012-05-22 | 7:44:00    | F9 v1.0 B0005   | CCAFS LC-40 | Dragon demo flight C2                                         | 525              | LEO (ISS) | NASA (COTS)     | Success         | No att          |
| 2012-10-08 | 0:35:00    | F9 v1.0 B0006   | CCAFS LC-40 | SpaceX CRS-1                                                  | 500              | LEO (ISS) | NASA (CRS)      | Success         | No att          |
| 2013-03-01 | 15:10:00   | F9 v1.0 B0007   | CCAFS LC-40 | SpaceX CRS-2                                                  | 677              | LEO (ISS) | NASA (CRS)      | Success         | No att          |

# Total Payload Mass

---

We calculated the total payload carried by boosters from NASA as 45596 using the query below

```
In [30]: %sql SELECT SUM(PAYLOAD_MASS_KG_) FROM SPACEXTBL WHERE customer = 'NASA (CRS)';
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
Out[30]: SUM(PAYLOAD_MASS_KG_)
```

```
45596
```

# Average Payload Mass by F9 v1.1

---

We calculated the average payload mass carried by booster version F9 v1.1 as 2928.4

```
In [32]: %sql SELECT AVG(PAYLOAD_MASS_KG_) FROM SPACEXTBL WHERE Booster_Version = 'F9 v1.1';
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[32]: AVG(PAYLOAD_MASS_KG_)  
          2928.4
```

# First Successful Ground Landing Date

---

We use the min() function to find the result

We observed that the dates of the first successful landing outcome on ground pad was 22<sup>nd</sup> December 2015

```
In [33]: %sql SELECT MIN(date) FROM SPACEXTBL WHERE Landing_Outcome = 'Success (ground pad)'
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[33]: MIN(date)  
         _____  
         2015-12-22
```

## Successful Drone Ship Landing with Payload between 4000 and 6000

---

We used the **WHERE** clause to filter for boosters which have successfully landed on drone ship and applied the **AND** condition to determine successful landing with payload mass greater than 4000 but less than 6000

```
In [38]: %sql SELECT Booster_Version FROM SPACEXTBL WHERE Landing_Outcome = 'Success (drone ship)' AND PAYLOAD_MASS_KG_ BETWEEN 4000
```

```
* sqlite:///my_data1.db  
Done.
```

```
Out[38]: Booster_Version
```

```
F9 FT B1022
```

```
F9 FT B1026
```

```
F9 FT B1021.2
```

```
F9 FT B1031.2
```



# Total Number of Successful and Failure Mission Outcomes

---

We used Groupby Mission\_Outcome and Count.  
100 Successful Outcomes and 1 Failure Outcome.

```
In [17]: %sql SELECT MISSION_OUTCOME, COUNT(*) as total_number \
          FROM SPACEXTBL \
          GROUP BY MISSION_OUTCOME;
```

```
* sqlite:///my_data1.db
Done.
```

```
Out[17]:
```

| Mission_Outcome                  | total_number |
|----------------------------------|--------------|
| Failure (in flight)              | 1            |
| Success                          | 98           |
| Success                          | 1            |
| Success (payload status unclear) | 1            |

# Boosters Carried Maximum Payload

```
In [18]: %sql SELECT BOOSTER_VERSION \
FROM SPACEXTBL \
WHERE PAYLOAD_MASS_KG_ = (SELECT MAX(PAYLOAD_MASS_KG_) FROM SPACEXTBL);
```

```
* sqlite:///my_data1.db
Done.
```

```
Out[18]: Booster_Version
```

|               |
|---------------|
| F9 B5 B1048.4 |
| F9 B5 B1049.4 |
| F9 B5 B1051.3 |
| F9 B5 B1056.4 |
| F9 B5 B1048.5 |
| F9 B5 B1051.4 |
| F9 B5 B1049.5 |
| F9 B5 B1060.2 |
| F9 B5 B1058.3 |
| F9 B5 B1051.6 |
| F9 B5 B1060.3 |
| F9 B5 B1049.7 |

We determined the booster that have carried the maximum payload using a subquery in the **WHERE** clause and the **MAX()** function.

# 2015 Launch Records

---

We used a combinations of the **WHERE** clause, **LIKE**, **AND**, and **BETWEEN** conditions to filter for failed landing outcomes in drone ship, their booster versions, and launch site names for year 2015

```
In [23]: %sql SELECT substr(Date,6,2) as month, DATE,BOOSTER_VERSION, LAUNCH_SITE, [Landing_Outcome] \
FROM SPACEXTBL \
where [Landing_Outcome] = 'Failure (drone ship)' and substr(Date,0,5)='2015';
```

```
* sqlite:///my_data1.db
Done.
```

```
Out[23]:
```

|  | month | Date       | Booster_Version | Launch_Site | Landing_Outcome      |
|--|-------|------------|-----------------|-------------|----------------------|
|  | 01    | 2015-01-10 | F9 v1.1 B1012   | CCAFS LC-40 | Failure (drone ship) |
|  | 04    | 2015-04-14 | F9 v1.1 B1015   | CCAFS LC-40 | Failure (drone ship) |

# Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

```
In [29]: %sql SELECT [Landing_Outcome], count(*) as count_outcomes \
FROM SPACEXTBL \
WHERE DATE between '2010-06-04' and '2017-03-20' group by [Landing_Outcome] order by count_outcomes DESC;
```

```
* sqlite:///my_data1.db
Done.
```

```
Out[29]:
```

| Landing_Outcome        | count_outcomes |
|------------------------|----------------|
| No attempt             | 10             |
| Success (drone ship)   | 5              |
| Failure (drone ship)   | 5              |
| Success (ground pad)   | 3              |
| Controlled (ocean)     | 3              |
| Uncontrolled (ocean)   | 2              |
| Failure (parachute)    | 2              |
| Precluded (drone ship) | 1              |

We selected Landing outcomes and the **COUNT** of landing outcomes from the data and used the **WHERE** clause to filter for landing outcomes **BETWEEN** 2010-06-04 to 2017-03-20.

We applied the **GROUP BY** clause to group the landing outcomes and the **ORDER BY** clause to order the grouped landing outcome in descending order.

A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The image is a composite of a solid blue background on the left and a satellite photograph of Earth on the right. The Earth's surface is dark, with numerous bright yellow and orange lights representing cities and urban areas. The horizon of the Earth is visible as a curved line separating the dark surface from the deep blue of space.

Section 3

# Launch Sites Proximities Analysis

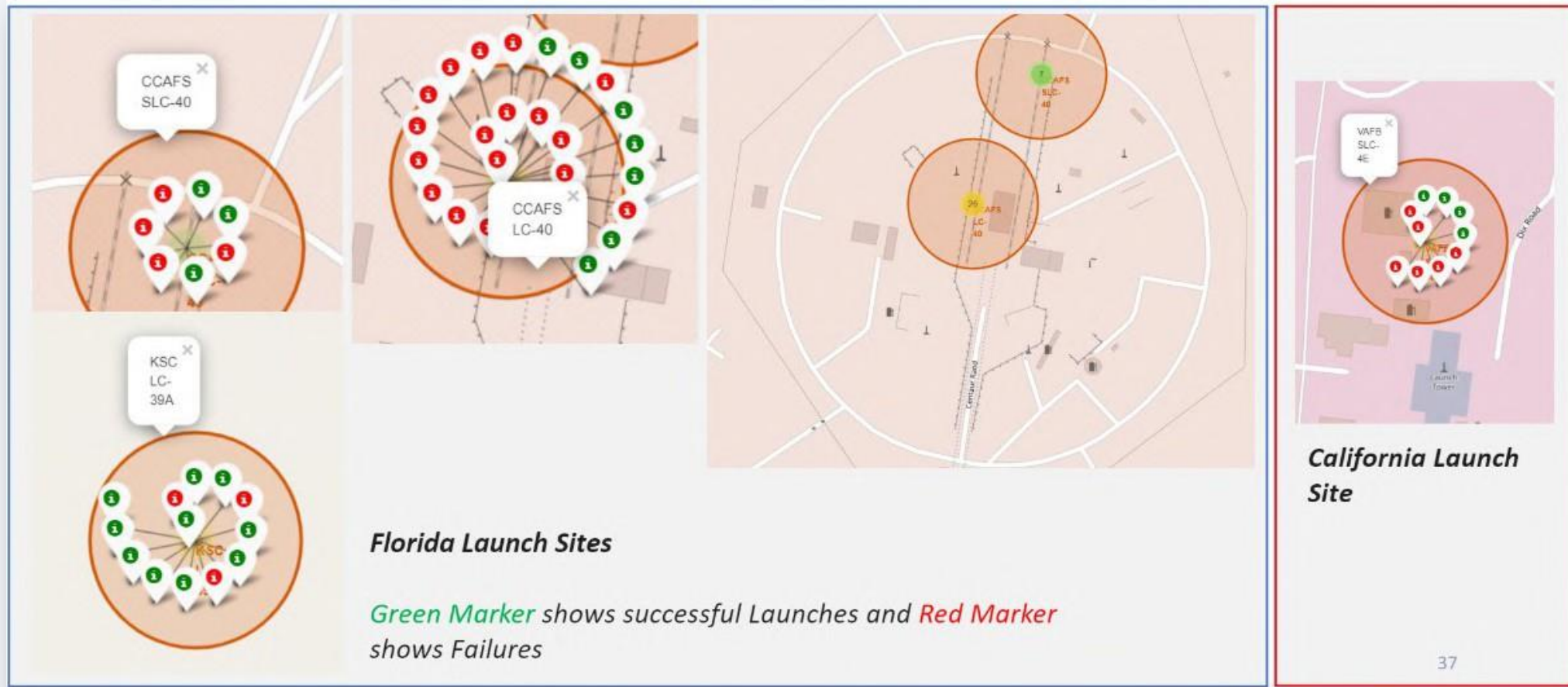
# Location of all the Launch Sites



We can see that all the SpaceX launch sites are located inside the United States

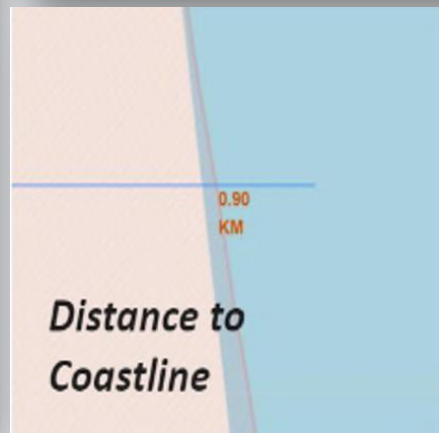
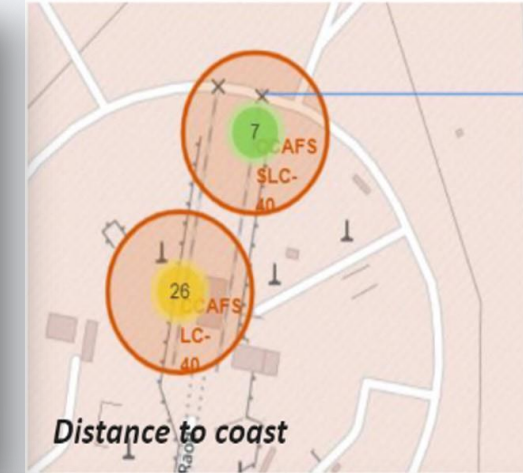
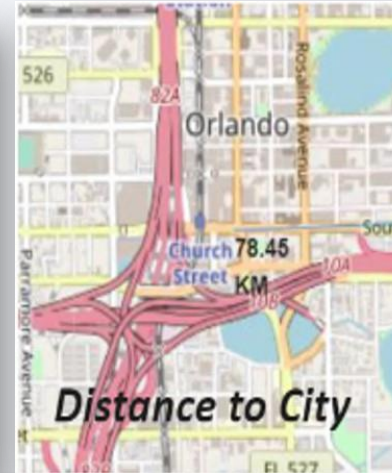
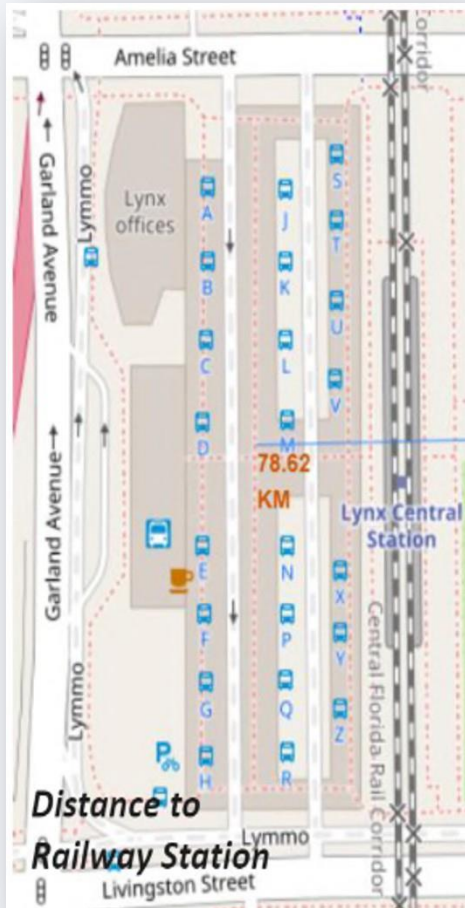


# Markers showing launch sites with color labels

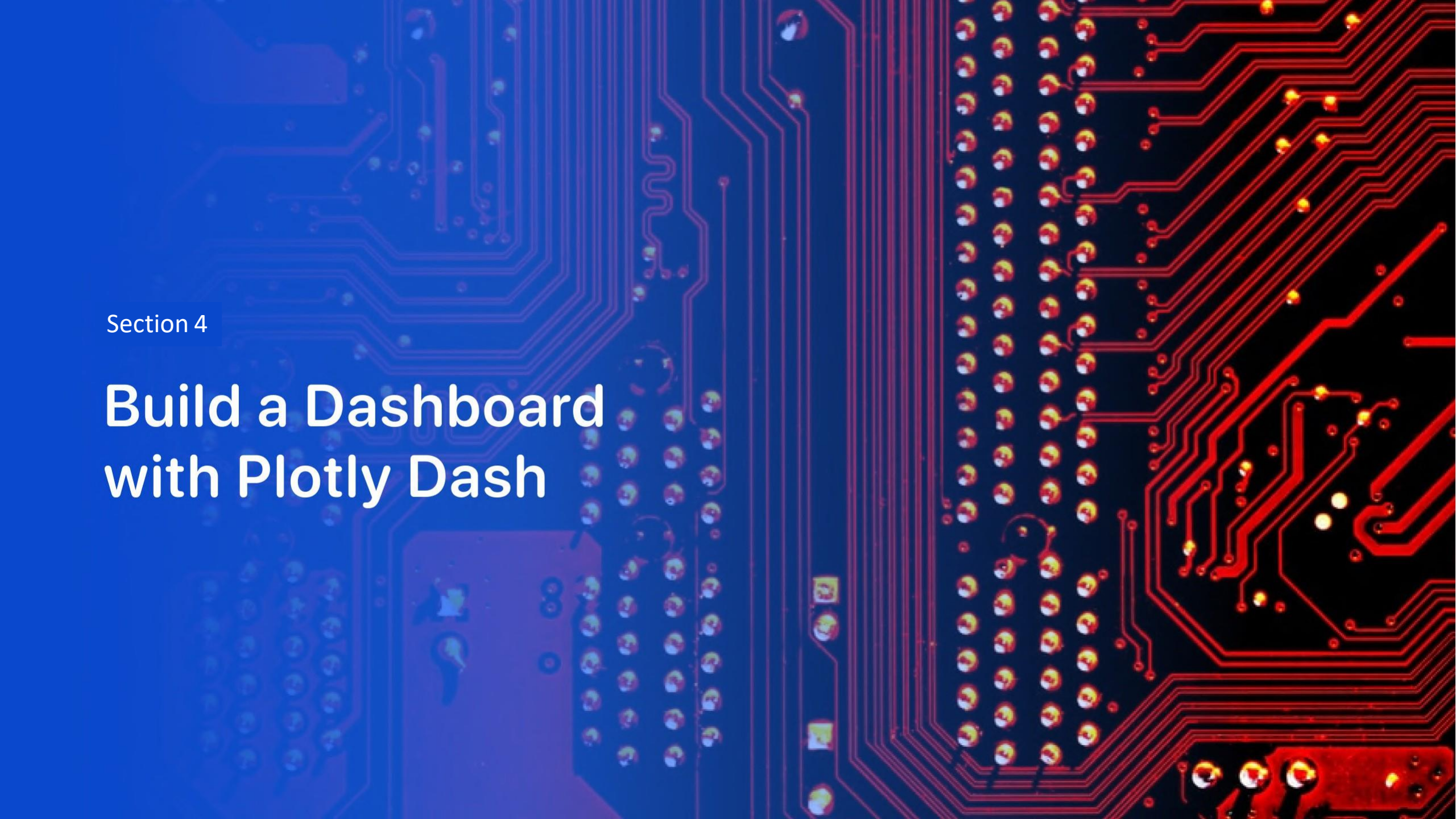




# Launch Sites Distance to Landmarks



- Are launch sites in close proximity to railways? No
- Are launch sites in close proximity to highways? No
- Are launch sites in close proximity to coastline? Yes
- Do launch sites keep certain distance away from cities? Yes



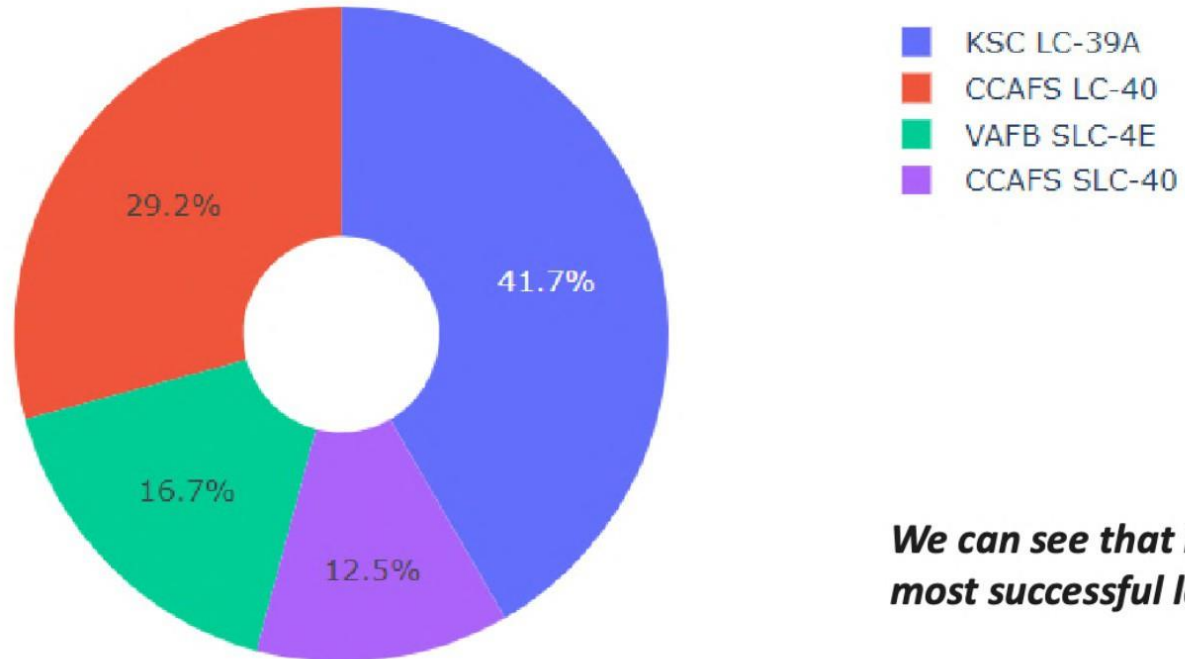
Section 4

# Build a Dashboard with Plotly Dash



# The success percentage by each sites.

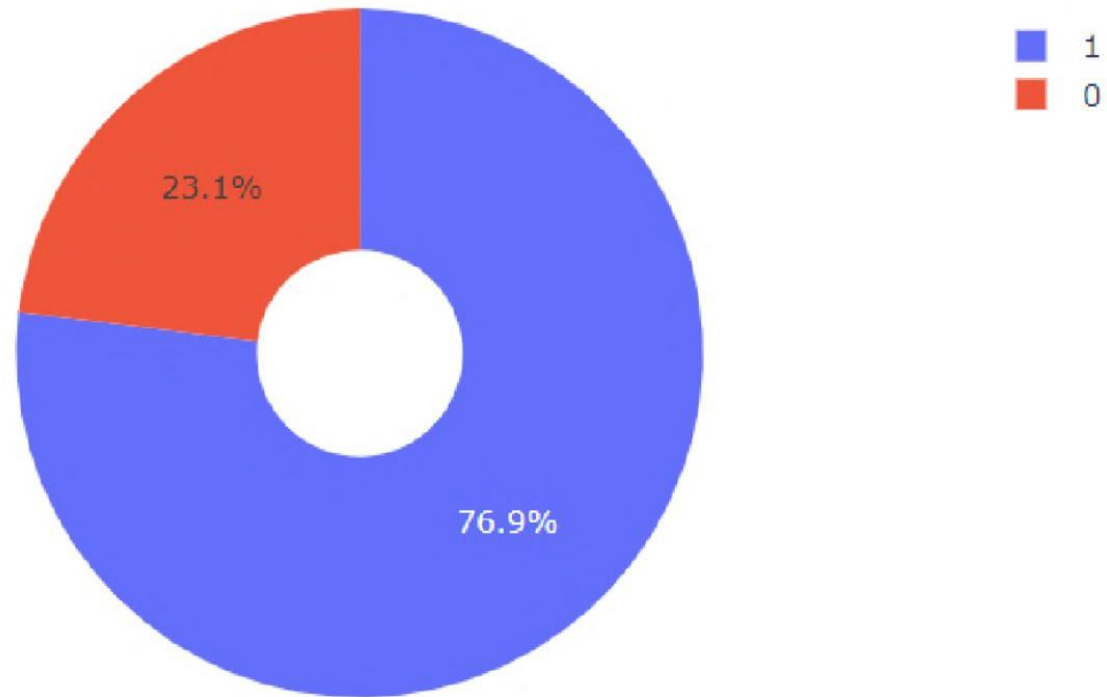
---



***We can see that KSC LC-39A had the most successful launches from all the sites***

# The highest launch-success ratio: KSC LC-39A

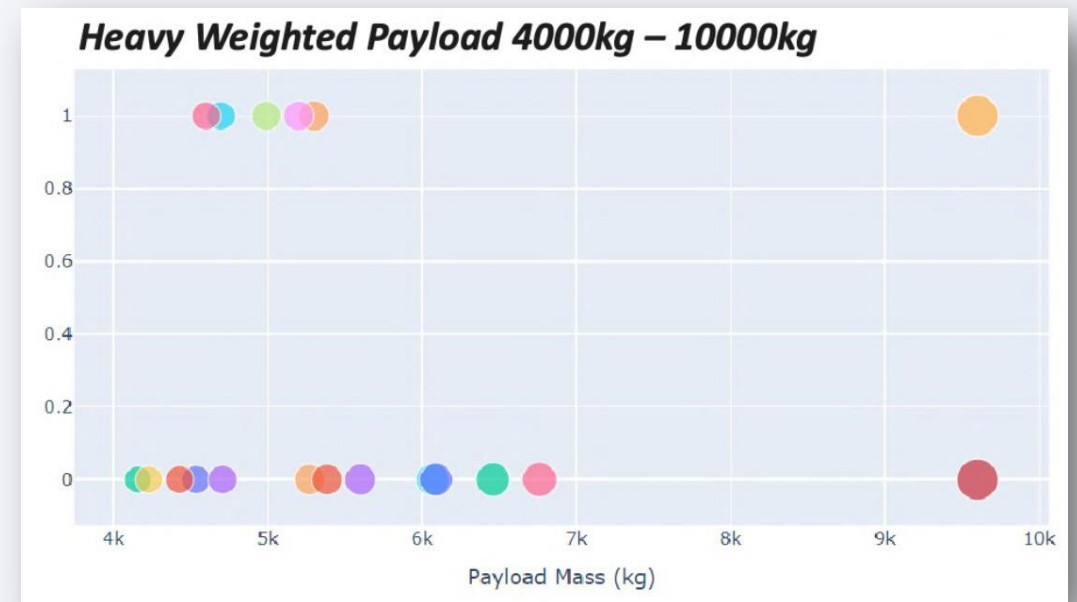
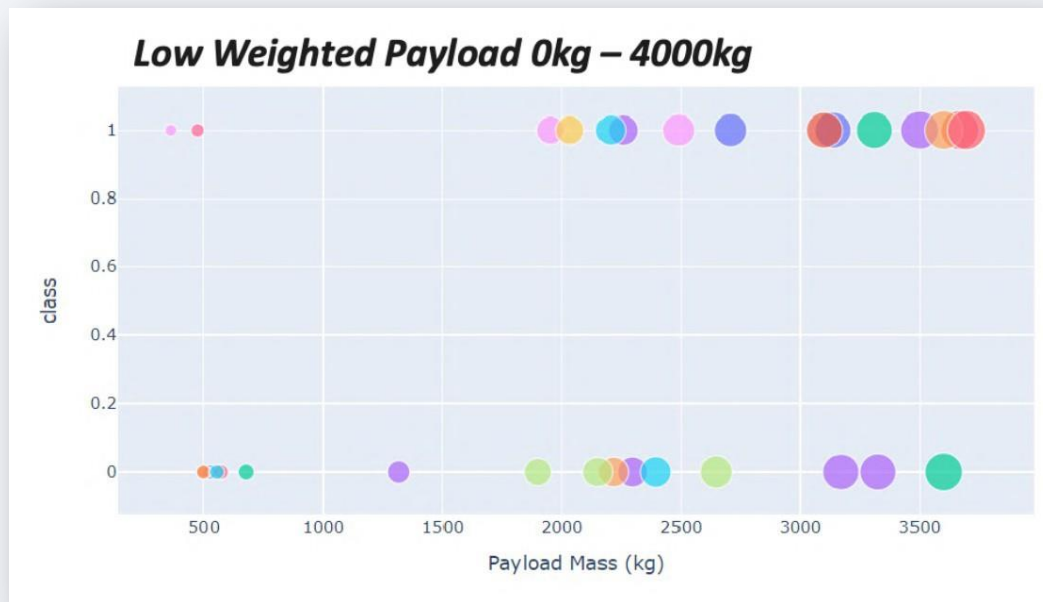
---



***KSC LC-39A achieved a 76.9% success rate while getting a 23.1% failure rate***

# Payload vs Launch Outcome Scatter Plot

We can see that all the success rate for low weighted payload is higher than heavy weighted payload





Section 5

# Predictive Analysis (Classification)

# Classification Accuracy

---

As we can see, by using the code as below: we could identify that the best algorithm to be the Decision Tree Algorithm which have the highest classification accuracy.

```
models = {'KNeighbors': knn_cv.best_score_,
          'DecisionTree': tree_cv.best_score_,
          'LogisticRegression': logreg_cv.best_score_,
          'SupportVector': svm_cv.best_score_}

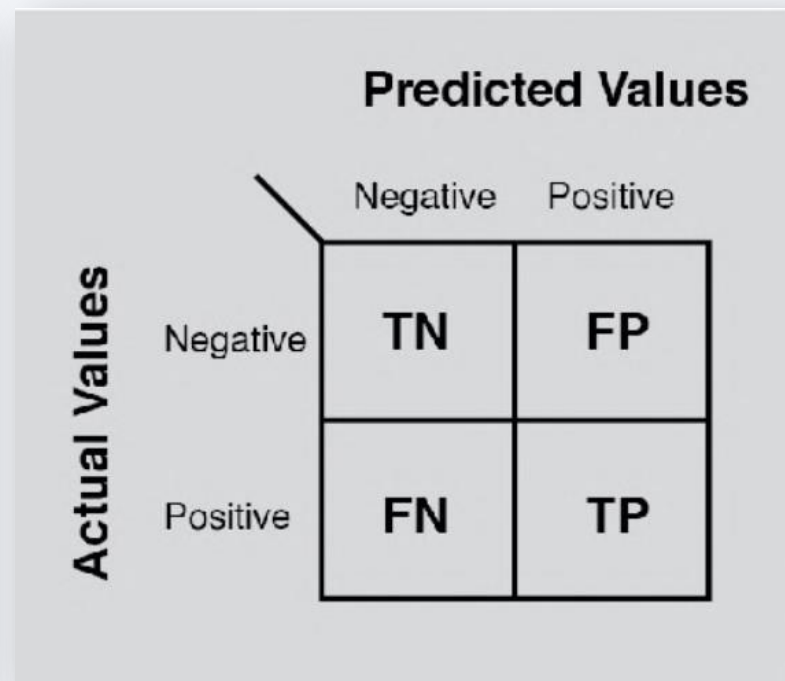
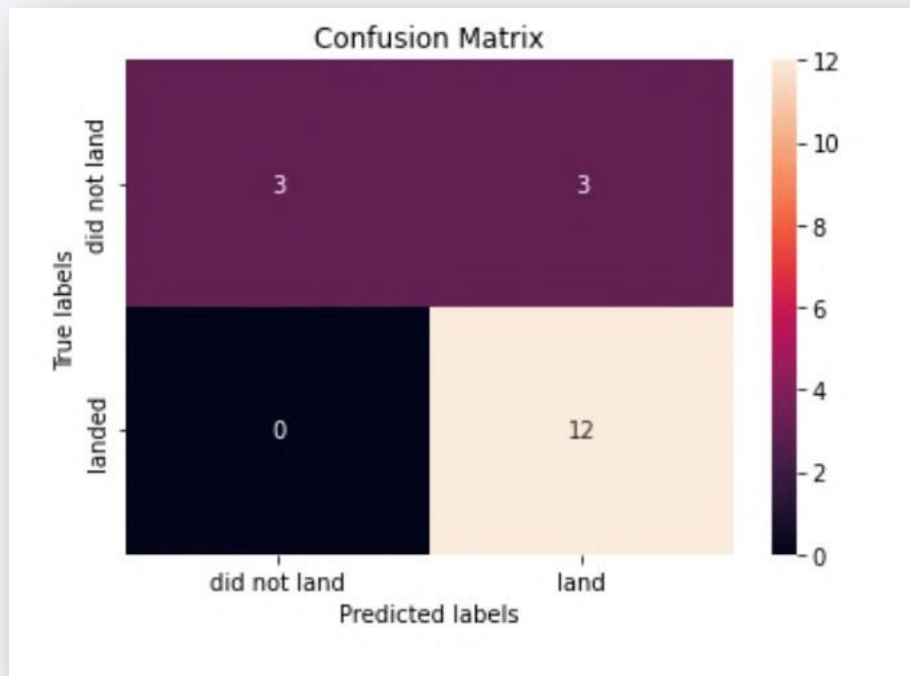
bestalgorithm = max(models, key=models.get)
print('Best model is', bestalgorithm, 'with a score of', models[bestalgorithm])
if bestalgorithm == 'DecisionTree':
    print('Best params is :', tree_cv.best_params_)
if bestalgorithm == 'KNeighbors':
    print('Best params is :', knn_cv.best_params_)
if bestalgorithm == 'LogisticRegression':
    print('Best params is :', logreg_cv.best_params_)
if bestalgorithm == 'SupportVector':
    print('Best params is :', svm_cv.best_params_)
```

```
Best model is DecisionTree with a score of 0.8857142857142856
Best params is : {'criterion': 'entropy', 'max_depth': 8, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 5, 'splitter': 'random'}
```



# Confusion Matrix

The confusion matrix for the decision tree classifier shows that the classifier can distinguish between the different classes. The major problem is the false positives .i.e., unsuccessful landing marked as successful landing by the classifier.



# Conclusions

---

We can conclude that:

- The Decision Tree Classifier Algorithm is the best Machine Learning approach for this dataset.
- The low weighted payloads (which define as 4000kg and below) performed better than the heavy weighted payloads.
- Starting from the year 2013, the success rate for SpaceX launches is increased, directly proportional time in years to 2020, which it will eventually perfect the launches in the future.
- KSC LC-39A have the most successful launches of any sites; 76.9%
- SSO orbit have the most success rate; 100% and more than 1 occurrence.

Thank you!

